
Chapter 11 Java Collections Framework

11.1 Lists

- Collections 
- `LinkedList` versus `ArrayList` 
- Iterators 
- Abstract Data Types (ADTs) 
- `LinkedList` Case Study: `Sieve` 

11.2 Sets

- Set Concepts 
- `TreeSet` versus `HashSet` 
- Set Operations 
- Set Case Study: `Lottery` 

11.3 Maps

- Basic Map Operations 

- **Map Views** (`keySet` and `values`) 
- `TreeMap` **versus** `HashMap` 
- **Map Case Study:** `WordCount` 
- **Collection Overview** 

Introduction

The previous chapter explored the `ArrayList` class. An `ArrayList` is one of many ways to store data in Java. In this chapter we'll explore Java's framework of collections, including lists, sets, and maps. We'll see how to use these structures together to manipulate and examine data in many ways to solve programming problems. This chapter will examine a trio of smaller interesting programs as case studies rather than presenting a unified case study at the end of the chapter.

We'll introduce a new type of list called a linked list that stores its data differently from an `ArrayList` but supports the same operations. We'll also discuss collections called sets that don't allow duplicate elements and that are easy

to search. Another collection type we'll explore is the map, which creates associations between pairs of data values. We'll also delve into the notion of abstract data types as a way to separate the capabilities of a collection from the details of its implementation.

11.1 Lists



The `ArrayList` class from [Chapter 10](#)  has several advantages over an array: It keeps track of its own size for you, it allows you to insert and remove data at arbitrary places in the array, and it resizes itself for you if it gets full.

In this section we'll learn about an object called a `LinkedList`, which is similar to an `ArrayList`. We'll also look at generalizing collections and discuss a useful object called an iterator that lets you examine the elements of any collection.

Collections

In [Chapters 7](#)  and [8](#) , we discussed ways to use arrays and classes to store data. The notion of organizing and structuring data

is an important one that helps us solve complex problems. Entities that store and manage data are also called *data structures*. Data structures can be used to implement sophisticated data storage objects called *collections*.

Collection

An object that stores a group of other objects, called its *elements*.

An `ArrayList` is an example of a collection. A collection uses a data structure internally to store its elements, such as an array or a set of objects that refer to one another. For example, an `ArrayList` is implemented using an array as its data structure, and a `TreeSet` (a collection introduced later in this chapter) is implemented using a data structure called a binary search tree.

Collections are categorized by the types of elements they store, the operations they allow you to perform on those elements, and the speed or efficiency of those operations. Here are some examples of collections:

- **List:** An ordered collection of elements, often accessed by integer indexes or by iteration.

- **Stack:** A collection in which the last element added is the first one to be removed.
- **Queue:** A collection in which elements are removed in the same order in which they were added.
- **Set:** A collection of elements that is guaranteed to contain no duplicates.
- **Map:** A collection of key/value pairs in which each key is associated with a corresponding value.

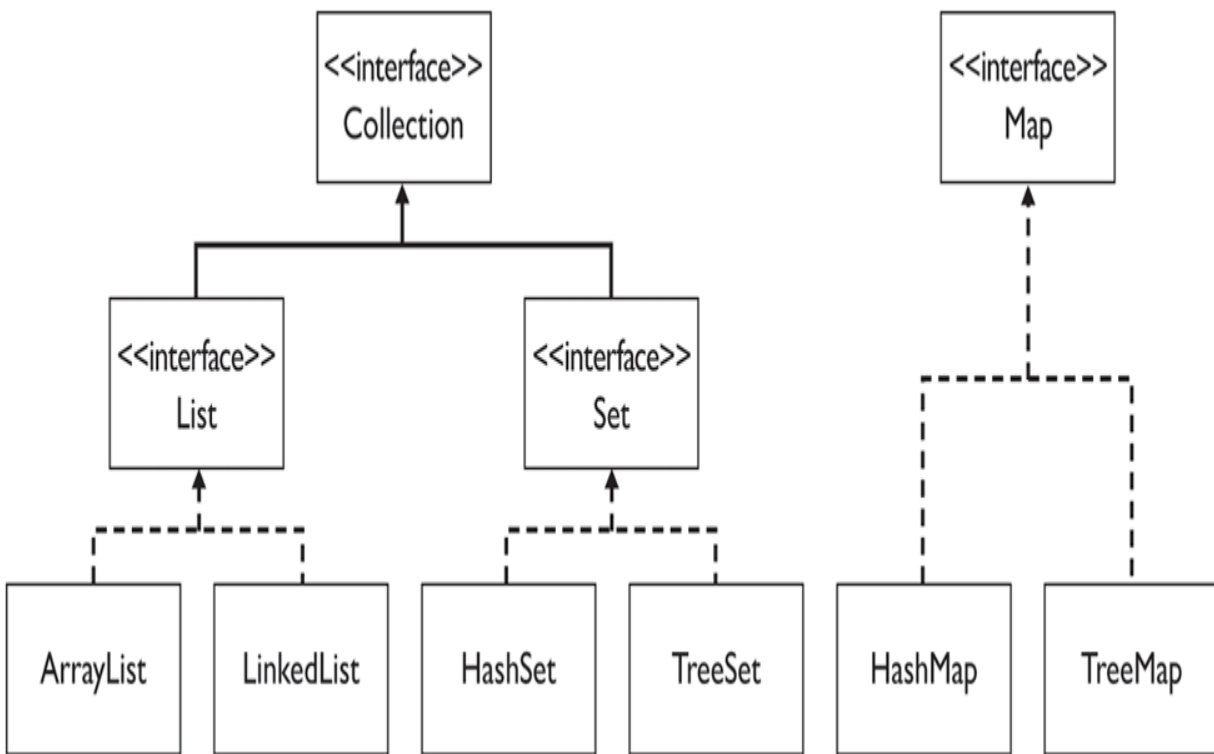
Java provides a large group of useful collections that allow you to store, access, search, sort, and manipulate data in a variety of ways. Together, these collections and classes are known as the *Java Collections Framework*. This framework is largely contained in the package `java.util`.

The `java.util` package contains an interface called `Collection` that is implemented by all collections except `Map`. This interface specifies the operations that most collections support. [Table 11.1](#)  lists those operations.

Table 11.1 Useful Methods of the `Collection` Interface

The `Collection` interface is extended and implemented by the other interfaces and classes in the Java Collections Framework. [Figure 11.1](#)  summarizes the various interfaces and the classes that implement them, all of which will be discussed in this chapter.

Figure 11.1 An abridged view of the Java Collections Framework



LinkedList versus ArrayList

Now we'll look at a collection called `LinkedList` and compare and contrast it with `ArrayList`.

`ArrayList` is a powerful and useful collection, but there are some cases in which using an `ArrayList` isn't ideal. For example, suppose we want to write a program to remove each `String` of even length from an `ArrayList` of `Strings`. We can do this by using a loop that

examines each element of the list and either removes it if its length is even or advances to the next string if its length is odd:

```
// Removes all strings of even length from the given list.
public static void removeEvenLength(ArrayList<String> list) {
    int i = 0;
    while (i < list.size()) {
        String element = list.get(i);
        if (element.length() % 2 == 0) {
            list.remove(i);
        } else {
            i++; // skip to next element
        }
    }
}
```

The preceding code is correct, but it doesn't perform well when the list has a lot of elements. On a relatively modern machine, it can take several minutes to process a list of a million elements. The reason it is so slow is that every time we remove an element from the list, we have to shift all subsequent elements to the left by one. This repeated shifting results in a slow program.

Another case in which an `ArrayList` behaves slowly is when it's used to model a waiting line or queue, where elements (customers) are always added to the end of the list (line) and always removed from the front. As customers arrive, they are added to the end of the list.

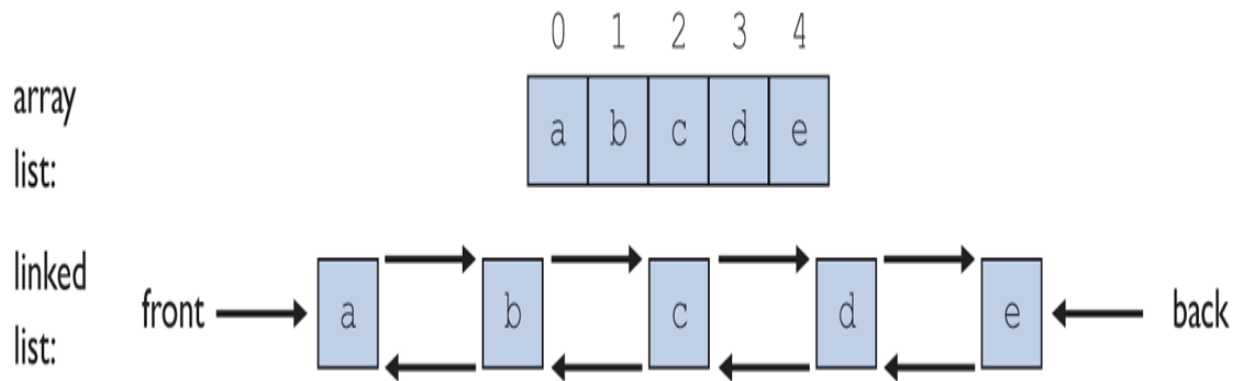
Customers are removed from the front of the list and processed in turn. Removing an element from the front of a large `ArrayList` is a slow operation because all the other elements have to be shifted to the left.

Another type of collection, called a *linked list*, can give better performance in problems like these that involve a lot of additions to or removals from the front or middle of a list. A linked list provides the same operations as an array list, such as `add`, `remove`, `isEmpty`, `size`, and `contains`. But a linked list stores its elements in a fundamentally different way. Elements of a linked list are stored in small individual containers called *nodes*. The nodes are “linked” together, each node storing a reference to the next node in the list. The overall linked list object keeps references to the front and back nodes.

Linked List

A collection that stores a list of elements in small object containers called *nodes*, which are linked together.

You can envision a linked list as an array list that’s been “broken apart,” with each element then stored in a small box (a node) connected to its neighboring box by an arrow.



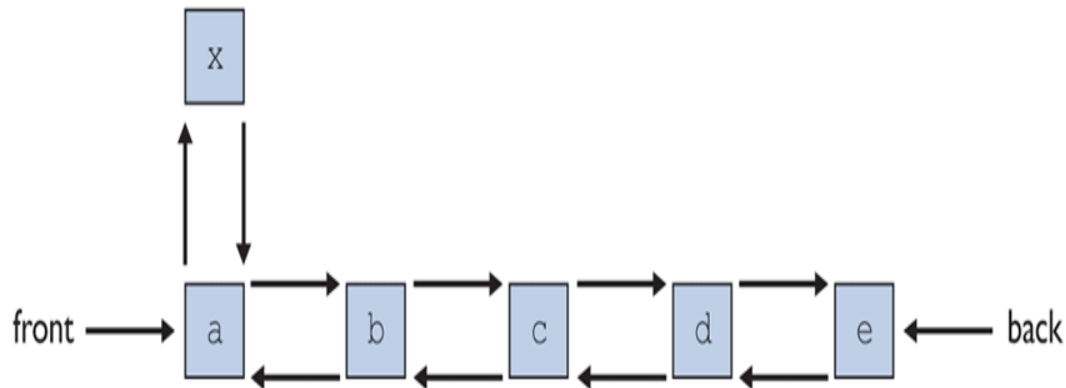
One major advantage of using a linked list is that elements can generally be added at the front of the list quickly, because rather than shifting all the elements in an array, the list just creates a new node object and links it with the others in the list. We don't have to do this ourselves; we simply call methods on the list, which takes care of it for us internally. **Figure 11.2**  shows what happens inside a linked list when an element is added at the front.

Figure 11.2 Adding an element to the front of a linked list

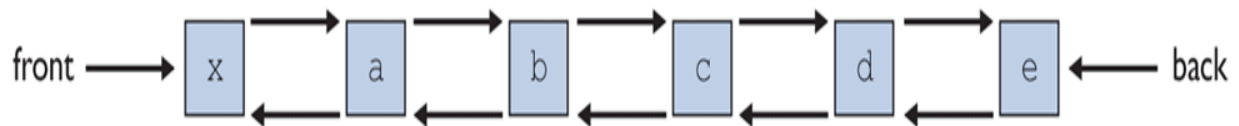
1. Make a new node to hold the new element.



2. Connect the new node to the other nodes in the list.



3. Change the front of the list to point to the new node.



To use a linked list in Java, create an object of type `LinkedList` instead of type `ArrayList`. A `LinkedList` object has the same methods you've used when working with `ArrayList`:

```
LinkedList<String> words = new LinkedList<>();  
words.add("hello");  
words.add("goodbye");  
words.add("this");  
words.add("that");
```

We could write a version of our `removeEvenLength` method that accepts a `LinkedList<String>` as its parameter rather than an `ArrayList<String>`. However, this change alone won't have much impact on performance. Since the `removeEvenLength` method examines each element of the list in sequential order, we can make the code more efficient by employing another type of object called an iterator.

Iterators

As we discussed in [Chapter 7](#) , arrays provide a nice property called *random access*, meaning that we can efficiently access or modify arbitrary array elements in any order. This access is possible because arrays are stored as large contiguous blocks of memory, so the computer can quickly compute the memory location of any of the array's elements. Linked lists, unfortunately, do not provide fast random access. A linked list is made up of many small node objects and generally keeps a direct reference only to one or two particular elements, such as the front and back elements. Therefore it is not possible to quickly access arbitrary elements of the list. A linked list is somewhat like a VHS tape or audio cassette tape in this way; if we wish to access an element, we must “fast-forward” or “rewind” through the list to the proper position.

When you call a method like `get`, `set`, `add`, or `remove` on a linked list, the code internally creates a temporary reference that begins at the front of the list and traverses the links between nodes until it reaches the desired index. The amount of time that this takes depends on the index you chose: If you ask for the element at index 5 it will be returned quickly, but if you ask for the element at index 9,000 the loop inside the `get` method must advance through 9,000 nodes, which will take much longer.

These methods tend to be slow when you use them on a linked list, especially if you call them many times or call them on a list with many elements. Imagine that after the preceding example call of `get(9000)` you decided to call `get(9001)`. The linked list doesn't remember its previous position, so it starts from the front again and advances 9,001 times. The only case in which the methods run quickly is when you pass an index near the front or back of the list.

Earlier in this chapter, we wrote a method to remove strings of even length from an `ArrayList`. If we adapted this code to use a `LinkedList` and made no other modifications we would find that it still runs very slowly on large lists, because it calls the `get` and `remove` methods many times:

```
// performs poorly on a linked list
public static void removeEvenLength(LinkedList<String> list) {
    int i = 0;
    while (i < list.size()) {
```

```
String element = list.get(i); // slow
if (element.length() % 2 == 0) {
    list.remove(i);           // slow
} else {
    i++;
}
}
```

However, there's an efficient way to examine every element of a linked list if we want sequential access (i.e., if we want to examine each element in order from the front to the back). To perform this task, we can use a special object called an *iterator* that keeps track of our current position in the list.

Iterator

An object that allows the efficient retrieval of the elements of a list in sequential order.

Using an iterator, when we move from one element to the next, we don't have to go back to the beginning of the list at each call and follow the links all the way from the front of the list to the desired index. As we'll see in this chapter, iterators are central to the Java

Collections Framework. Every collection provides iterators to access its elements. In other words, there's a familiar interface for examining the elements of any collection.

An iterator object has the methods listed in [Table 11.2](#). The methods `next` and `hasNext` should be familiar from the `Scanner` class that you have already studied, and their behavior is similar here.

Table 11.2 Methods of Iterator Objects

To get an iterator from most collections, such as an `ArrayList` or a `LinkedList`, you call the method `iterator` on the list, which returns an object of type `Iterator` for examining that list's elements. (You don't use the `new` keyword.) Generally, a variable named `list` storing elements of type `E` uses an iterator in the following way:

```
Iterator<E> itr = list.iterator();
while (itr.hasNext()) {
    <do something with itr.next()>;
}
```

The example of removing strings that have even length from a collection can be implemented much more efficiently using an iterator:

```
// removes all strings of even length from the given linked
list

public static void removeEvenLength(LinkedList<String> list) {
    Iterator<String> i = list.iterator();
    while (i.hasNext()) {
        String element = i.next();
        if (element.length() % 2 == 0) {
            i.remove();
        }
    }
}
}
```

Whereas the original `ArrayList` version required up to several minutes to process a list of one million elements on a modern computer, this new code finishes a million-element list in under one-tenth of a second. It performs so quickly because the iterator retains the current position in the list between calls to get or remove elements.

Iterators are also used internally by Java's for-each loop. When you use a for-each loop like the following, Java is actually accessing the elements using an iterator under the hood:

```
for (String word : list) {
    System.out.println(word + " " + word.length());
}
```

COMMON PROGRAMMING ERROR

Calling `next` on an Iterator Too Many Times

Iterators can be a bit confusing to new programmers, so you have to be careful to use them correctly. The following code attempts to use an iterator to find and return the longest string in a linked list, but it has a bug:

```
// returns the longest string in the list (does not
work!)
public static String longest(LinkedList<String> list) {
    Iterator<String> itr = list.iterator();

    String longest = itr.next(); // initialize to first
    element

    while (itr.hasNext()) {
        if (itr.next().length() > longest.length()) {
            longest = itr.next();
        }
    }

    return longest;
}
```

The problem with the previous code is that its loop calls the `next` method on the iterator in two places: once when it tests the length of the string, and again when it tries to store the string as the longest. Each time you call `next`, the iterator advances by one position, so if it's called twice in the loop, you'll skip an element when you find a match. For example, if the list contains the elements (`"oh"`, `"hello"`, `"how"`, `"are"`, `"you"`), the program might see the `"hello"` and intend to store it, but the second call to `next` would actually cause it to store the element that follows `"hello"`, namely, `"how"`.

The solution is to save the result of the `itr.next()` call into a variable. The following code would replace the `while` loop in the previous code:

```
// this version of the code is correct
while (itr.hasNext()) {
    String current = itr.next();
    if (current.length() > longest.length()) {
        longest = current;
    }
}
```

As the compiler processes the for-each loop, it essentially converts the loop into the following code:

```
Iterator<String> i = list.iterator();  
while (i.hasNext()) {  
    String word = i.next();  
    System.out.println(word + " " + word.length());  
}
```

There's a more advanced version of `Iterator` called `ListIterator` that works only on lists. A `ListIterator` provides operations like adding elements, setting element values, and reversing iteration from back to front. Because it is more complex, we won't discuss `ListIterator` in detail in this book. You can read more about it online in the [Java API](#) and [Java Tutorial](#) pages.

In summary, the following table compares the major benefits of `ArrayList` and `LinkedList`:

Java 8 has introduced some powerful new features for processing lists and other collections. The following Java 8 code would produce a new list that does not contain any even-length strings:

```
public static List<String> removeEvenLength(List<String> list)  
{  
    return list.stream()  
        .filter(s -> s.length() % 2 != 0)
```

```
.collect(Collectors.toList());  
}
```

You can read more about this approach in [Chapter 19](#) .

Abstract Data Types (ADTs)

It's no accident that the `LinkedList` collection provides the same methods as the `ArrayList`. They both implement the same kind of collection: a *list*. At a high level, the most important thing isn't the way the list is implemented internally, but the operations we can perform on it. This set of operations is an example of an *abstract data type*, or ADT.

Abstract Data Type (ADT)

A specification of a type of data and the operations that can be performed on it.

An ADT specifies operations that can be performed on data without specifying exactly how those operations are implemented. Linked lists and array lists are both examples of the list ADT because they

both provide the same operations, such as storing data by index, adding and removing data at particular indexes, and so on.

In Java, ADTs are specified by interfaces. Each ADT's operations are specified by the methods of its interface. For example, both `LinkedList` and `ArrayList` implement an interface in the `java.util` package called `List`. The `List` interface declares all the common methods that both types of lists implement.

It's a good practice to declare any variables and parameters of a collection type using the appropriate interface type for that ADT rather than the actual class's type. For example, the following code constructs a `LinkedList` object but stores it in a variable of type `List`:

```
List<Integer> list = new LinkedList<>();
```

Joshua Bloch, one of the authors of the Java Collections Framework, says that this programming practice is “strongly recommended” because “it gives you the flexibility to change implementations.” Note that you cannot create an object of type `List`, but you can use `List` as the type of a variable.

You can also use the interface types for ADTs like `List` when you declare parameters, return types, or fields. Doing so is useful when you're writing a method that accepts a collection as a parameter, because it means that the method will be able to operate

successfully on any collection that implements that ADT's interface. For example, the following method can accept a `LinkedList<String>` or an `ArrayList<String>` as its actual parameter:

```
// returns the longest string in the given list
// pre: list.size() > 0
public static String longest(List<String> list) {
    Iterator<String> i = list.iterator();
    String result = i.next();
    while (i.hasNext()) {
        String next = i.next();
        if (next.length() > result.length()) {
            result = next;
        }
    }
    return result;
}
```

It works with either type of list and is efficient for both. This flexibility is another benefit of polymorphism (discussed in [Chapter 9](#)). In fact, you could make the method even more general by having it accept a parameter of type `Collection` rather than `List`, since every collection has an `iterator` method.

The `java.util` package has a class called `Collections` that contains several useful methods related to all collections. (Note that this class

is not the same as the `Collection` interface that many collection classes implement.) The `Collections` class contains static methods that operate on lists. These methods' headers specify parameters of type `List` rather than `LinkedList` or `ArrayList`. The methods perform common tasks on lists, such as sorting, shuffling, and searching.

Table 11.3  presents a short list of useful methods from the `Collections` class that operate on lists.

Table 11.3 Useful Static Methods of the `Collections` Class

Notice that these methods are static, so they must be called by writing the word `Collections` followed by a dot and the method's name. For example, if you had a `LinkedList` variable called `list` and you wanted to reverse the list's contents, you'd write

```
Collections.reverse(list);
```

Since Java 8, the `List` interface contains some useful static methods for creating and manipulating lists. Perhaps the most useful is the `of` method, which takes element values as parameters and returns a fixed-length list containing those elements, similar to the `Arrays.asList` method shown in **Chapter 10** . This is a nice syntax for quickly creating a list of elements. The following JShell interaction creates such a list:

```
jshell> List<Integer> nums = List.of(10, 20, 30, 40);  
nums ==> [10, 20, 30, 40]  
  
jshell> nums.size()  
$2 ==> 4
```

The list returned by `List.of` is a special list that cannot be changed. Your program will crash if you try to modify the list:

```
jshell> nums.remove(0);  
| java.lang.UnsupportedOperationException thrown  
|       at AbstractList.remove (AbstractList.java:167)  
|       at (#3:1)
```

If you want a modifiable version of such a list, create a regular `LinkedList` or `ArrayList` and pass the result of `List.of` as the parameter to its constructor. Doing so will make a copy of the result of `List.of` that is able to be changed, as shown in the following JShell interaction:

```
jshell> List<Integer> nums2 = new LinkedList<>(  
...> List.of(10, 20, 30, 40));  
nums2 ==> [10, 20, 30, 40]  
  
jshell> nums2.remove(0);
```

```
$4 ==> 10  
  
jshell> nums2  
nums2 ==> [20, 30, 40]
```

There are similar methods `Set.of` and `Map.of` for the `Set` and `Map` collections we'll see later in this chapter.

In addition to `List`, there are several other interfaces, such as `Queue`, `Set`, and `Map`, representing ADTs in the Java Collections Framework. We'll explore several of them in this chapter.

LinkedList Case Study: Sieve

Consider the task of finding all prime numbers up to a given maximum. Prime numbers are integers that have no factors other than 1 and themselves. The number 2 is the smallest prime number.

To build a list of prime numbers, you could just write a brute-force solution using `for` loops:

```
for (each number from 2 to maximum) {  
    if (number is prime) {  
        add number to list of prime numbers.  
    }  
}
```

```
}  
  
}
```

But you would need a way to figure out whether each number is prime. One option would be to write another `for` loop that tested all lower integers to see whether they were factors of that number. However, there's an easier way.

The *sieve of Eratosthenes*, named for the Ancient Greek mathematician who devised it, is a classic algorithm for finding prime numbers. The sieve algorithm starts by creating two lists of numbers: one list of numbers to process (some of which may be prime), and another list of numbers known to be prime. Initially, the list of numbers to process can contain every number from 2 to the maximum, while the list of primes will be empty. Here are the initial two lists for a maximum of 25:

```
numbers: [2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,  
17, 18, 19,  
20, 21, 22, 23, 24, 25]  
primes: []
```

The sieve algorithm begins by removing the first element from the `numbers` list and adding it to the `primes` list. This number is found to be prime because of the nature of the algorithm. Next, the algorithm filters out all the other elements from the `numbers` list that are

multiples of this prime number. On the first pass of the algorithm, for example, the program selects 2 from the `numbers` list, places it into the `primes` list, and removes all multiples of 2 from the `numbers` list. Now the number at the front of the `numbers` list is 3. This number will be placed into the `primes` list during the next pass of the algorithm, and all the multiples of 3 that appear in the `numbers` list will be removed.

The numbers taken from the front of the `numbers` list are guaranteed to be prime. A nonprime number cannot reach the front of the `numbers` list because every nonprime number must be a multiple of some prime number, and any such multiples will have been removed by a previous pass of the algorithm.

Here are the two lists after the first three passes of the algorithm:

```
numbers: [3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25]
primes:  [2]

numbers: [5, 7, 11, 13, 17, 19, 23, 25]
primes:  [2, 3]

numbers: [7, 11, 13, 17, 19, 23]
primes:  [2, 3, 5]
```

Now let's implement the sieve algorithm. We'll use `LinkedLists` to represent the lists of numbers and primes. This tool is preferable to `ArrayLists` because, as we discussed previously, removing elements from the front of an `ArrayList` is inefficient.

First we'll create an empty list of primes and a list of all the numbers up to the given maximum. Since we've discussed ADTs and the `List` interface, we'll declare our variables as the ADT interface type `List<Integer>`:

```
List<Integer> primes = new LinkedList<>();
List<Integer> numbers = new LinkedList<>();
for (int i = 2; i <= max; i++) {
    numbers.add(i);
}
```

Next, we'll process the list of numbers. We'll use an iterator to make passes over the `numbers` list and remove elements that are multiples of the `front` element:

```
while (!numbers.isEmpty()) {
    // remove a prime number from the front of the list
    int front = numbers.remove(0);
    primes.add(front);
    // remove all multiples of this prime number
```

```

        Iterator<Integer> itr = numbers.iterator();
        while (itr.hasNext()) {
            int current = itr.next();
            if (current % front == 0) {
                itr.remove();
            }
        }
    }
}

```

The lines of code that follow are the complete program. The most significant addition is a `main` method that prompts the user for the maximum number:

```

1  // Uses a linked list to implement the sieve of
2  // Eratosthenes algorithm for finding prime numbers.
3
4  import java.util.*;
5
6  public class Sieve {
7      public static void main(String[] args) {
8          System.out.println("This program will tell you all
prime");
9          System.out.println("numbers up to a given
maximum.");
10         System.out.println();
11
12         Scanner console = new Scanner(System.in);

```

```
13         System.out.print("Maximum number? ");
14         int max = console.nextInt();
15
16         List<Integer> primes = sieve(max);
17         System.out.println("Prime numbers up to " + max +
18 ":"");
19         System.out.println(primes);
20     }
21
22     // Returns a list of all prime numbers up to given max
23     // using the sieve of Eratosthenes algorithm.
24     public static List<Integer> sieve(int max) {
25         List<Integer> primes = new LinkedList<>();
26
27         // add all numbers from 2 to max to a list
28         List<Integer> numbers = new LinkedList<>();
29         for (int i = 2; i <= max; i++) {
30             numbers.add(i);
31         }
32
33         while (!numbers.isEmpty()) {
34             // remove a prime number from the front of the
35             list
36             int front = numbers.remove(0);
37             primes.add(front);
38
39             // remove all multiples of this prime number
40             Iterator<Integer> itr = numbers.iterator();
```

```

39         while (itr.hasNext()) {
40             int current = itr.next();
41             if (current % front == 0) {
42                 itr.remove();
43             }
44         }
45     }
46
47     return primes;
48 }
49 }

```

The following is a sample log of execution of the program:

```

This program will tell you all prime
numbers up to a given maximum.
Maximum number? 50
Prime numbers up to 50:
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

```

Our version of the sieve algorithm has been simplified. The real algorithm stops when the first element of the `numbers` list is greater than the square root of the maximum, because any number this large that remains in the list cannot have any multiples remaining in the list. For example, when the maximum is 25, once the item at the

front of the `numbers` list exceeds 5, all remaining numbers in the list are known to be prime and can be placed in the `primes` list.

The algorithm can be improved in other ways. For example, the initial list of numbers doesn't need to store every integer from 2 through the maximum. It can instead store 2 and each odd integer up to the maximum, because no other even numbers are prime. This modification makes the algorithm more efficient because fewer numbers need to be processed. These two improvements to the algorithm are left as an exercise.

11.2 Sets



A major limitation of both linked and array lists is that searching them takes a long time. Generally, if you want to search a list, you have to look at each element sequentially to see whether you've found the target. This can take a long time for a large list.

Another limitation of lists is that it's not easy to prevent a list from storing duplicate values. In many cases this isn't a problem, but if, for example, you are storing a collection to count the number of unique words in a book, you don't want any duplicates to exist. To prevent duplicates in a list, you have to sequentially search the list every time you add to it, in order to make sure you aren't adding a word that's already present.

When you want to maintain a collection of elements that can be searched quickly and that prevents duplicates, you're better off using another abstract data type called a *set*.

Set

A collection that cannot contain duplicates.

The `Set` collection is very much like the mathematical notion of a set. Sets do not support all the operations you can perform on lists (namely, any operation that requires an index), but they do offer the benefits of fast searching and effortless elimination of duplicates.

Set Concepts

The two primary implementations of the Java Collections Framework's `Set` interface are called `HashSet` and `TreeSet`. `HashSet` is the general-purpose set class, whereas `TreeSet` offers a few advantages that will be discussed later. If you wanted to store a set of `String` values, you could write code like the following:

```
Set<String> stooges = new HashSet<>();  
stooges.add("Larry");  
stooges.add("Moe");  
stooges.add("Curly");
```

```
stooges.add("Moe"); // duplicate, won't be added
stooges.add("Shemp");
stooges.add("Moe"); // duplicate, won't be added
```

After the code executes, the set will contain only four elements, because "Moe" will be placed into the set only once. Notice that, as with lists, you can declare your collection variable to be of the interface type rather than the class type (type `Set` rather than type `HashSet`).

A `Set` provides all of the operations from the `Collection` interface introduced earlier in this chapter, such as `add`, `contains`, and `remove`. It's generally assumed that the `Set` performs these operations efficiently, so you can add many elements to a `Set` and search it many times without experiencing poor performance. A `Set` also provides a `toString` method that lets you see its elements. Printing the preceding `stooges` set would produce the following output:

```
[Moe, Shemp, Larry, Curly]
```

One of the most important benefits of using a `Set` such as `HashSet` is that it can be searched incredibly quickly. Recall that the `contains` method of an `ArrayList` or `LinkedList` must examine every element of the list in order until it finds the target value. By contrast, the `contains` method of a `HashSet` is implemented in such a way that it

often needs to examine just one element, making it a much more efficient operation.

A `HashSet` is implemented using a special internal array called a *hash table* that places elements into specific positions based upon integers called *hash codes*. (Every Java object has a hash code that can be accessed through its `hashCode` method.) You don't need to understand the details of `HashSet`'s implementation to use it—the bottom line is that it's implemented in such a way that you can add, remove, and search for elements very quickly. We will discuss hash set implementations in detail in [Chapter 18](#) .

One drawback of `HashSet` is that it stores its elements in an unpredictable order. The elements of the `stooges` set were not alphabetized, nor did they match the order in which they were inserted. This storage practice is a tradeoff for the `HashSet`'s fast performance. (Java now includes a variation called `LinkedHashSet` that is almost as fast as `HashSet` but stores its elements in the order they were inserted.)

Sets allow you to examine lots of data while ignoring duplicates. For example, if you wanted to see how many unique words appear in the book *Moby-Dick*, you could write code such as the following:

```
Set<String> words = new HashSet<>();  
Scanner in = new Scanner(new File("mobydick.txt"));  
while (in.hasNext()) {
```

```
String word = in.next();  
word = word.toLowerCase();  
words.add(word);  
}  
System.out.println("Number of unique words = " + words.size());
```

This code produces the following output when run on the text of *Moby-Dick* (available from <http://www.gutenberg.org>):

```
Number of unique words = 30368
```

The `HashSet` class has a convenient constructor that accepts another collection as a parameter and puts all the unique elements from that collection into the `Set`. One clever usage of this constructor is to find out whether a `List` contains any duplicates. To do so, simply construct a `HashSet` from the list and see whether the sizes differ:

```
// returns true if the given list contains any duplicate  
elements  
public static boolean hasDuplicates(List<Integer> list) {  
    Set<Integer> set = new HashSet<>(list);  
    return set.size() < list.size();  
}
```

One drawback of a `Set` is that it doesn't store elements by indexes. The following loop doesn't compile on a `Set`, because it doesn't have a `get` method:

```
// this code does not compile
for (int i = 0; i < words.size(); i++) {
    String word = words.get(i); // error -- no get method
    System.out.println(word);
}
```

Instead, if you want to loop over the elements of a `Set`, you must use an iterator. Like other collections, a `Set` has an `iterator` method that creates an `Iterator` object to examine its elements. You can then use the familiar `hasNext/next` loop to examine each element:

```
// this code works correctly
Iterator<String> itr = words.iterator();
while (itr.hasNext()) {
    String word = itr.next();
    System.out.println(word);
}
```

A shorter alternative to the preceding code is to use a for-each loop over the elements of the `set`. As mentioned previously, the code

behaves the same way but is easier to write and read than the version with the `while` loop:

```
for (String word : words) {  
    System.out.println(word);  
}
```

TreeSet versus HashSet

The examples in the preceding section used `HashSet`, but there's another class called `TreeSet` that also implements the `Set` interface. A `TreeSet` uses an internal linked data structure called a *binary search tree* to store its elements in sorted order. We will discuss binary tree implementations in detail in [Chapter 17](#). A `TreeSet` is efficient for adding, removing, and searching, though it is a bit slower than a `HashSet`.

A `TreeSet` can be useful if you want to print the set and have the output ordered. For example, the following code displays the sorted set of all three-letter words in *Moby Dick* that start with “a”:

```
Set<String> words = new TreeSet<>();  
Scanner in = new Scanner(new File("mobydick.txt"));
```

```

while (in.hasNext()) {
    String word = in.next();
    word = word.toLowerCase();
    if (word.startsWith("a") && word.length() == 3) {
        words.add(word);
    }
}

System.out.println("Three-letter 'a' words = " + words);

```

The code produces the following output:

```

Three-letter 'a' words = [act, add, ado, aft, age, ago, ah!,
ah,, aid, aim, air, alb, ale, ali, all, am,, am-, am:, and,
ant, any, apt, arc, are, ark, arm, art, as,, as-, as., ash,
ask, ass, at,, at., at;, at?, ate, awe, axe, aye]

```

A `TreeSet` can be used with data that has a natural ordering. This means that it will work if its elements are of any type that implements the `Comparable` interface, such as `Integer` or `String`. You can also provide your own object that specifies how to compare elements, called a *comparator*. Comparators will be discussed in [Chapter 13](#)  when we cover searching and sorting.

You should not try to construct a `TreeSet` of objects without a natural ordering, such as `Point` objects:

```
// this code compiles but will lead to a runtime error  
Set<Point> points = new TreeSet<>();
```

The preceding code compiles (unfortunately), but it generates an exception when you run it because it doesn't know how to order the `Point` objects in the `TreeSet`:

```
Exception in thread "main"  
    java.lang.ClassCastException: java.awt.Point  
        at java.util.TreeMap.compare(Unknown Source)  
        at java.util.TreeMap.put(Unknown Source)  
        at java.util.TreeSet.add(Unknown Source)
```

You'd be better off using a `HashSet` in this case.

In summary, the following are some of the major differences between `HashSet` and `TreeSet`:

Set Operations

Consider the task of figuring out how many unique elements appear in two given sets. You cannot just add the sets' sizes, since they

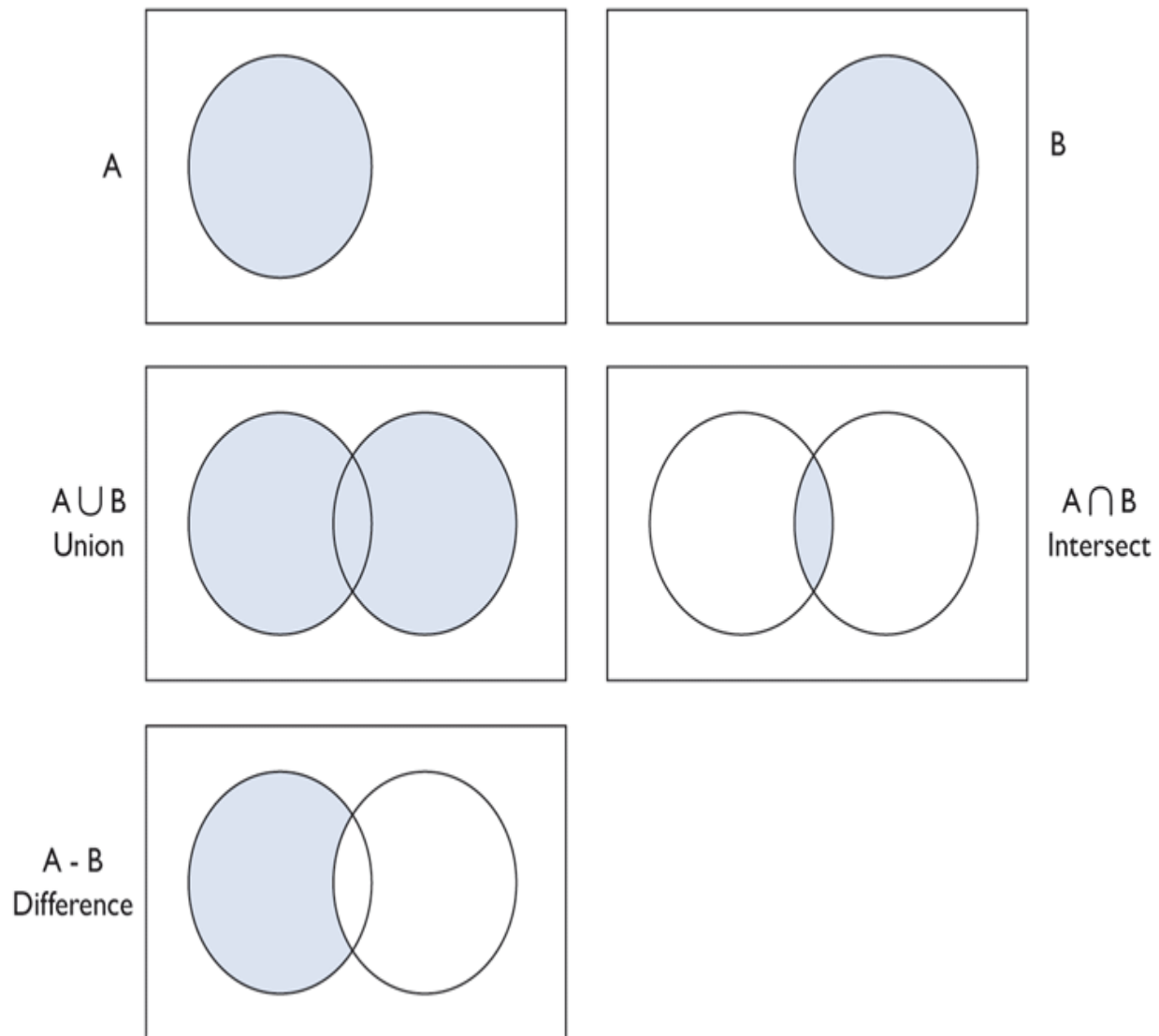
might have some elements in common that should not be counted twice in your total. Instead, you could count all elements from the first set and then count only the unique elements of the second, by checking to see whether each element from the second is also in the first:

```
// Returns the number of unique elements contained
// in either set1 or set2. Not a good model to follow.
public static int totalElements(Set<String> set1, Set<String>
set2) {
    int count = set1.size();
    for (String element : set2) {
        if (!set1.contains(element)) {
            count++;
        }
    }
    return count;
}
```

However, a more elegant way to perform this calculation is to compute a *union* between the sets. The union of two sets A and B is the set of all elements contained in either A, B, or both. Union is an example of a *set operation*. Other examples of set operations are *intersection* (the set of all elements that are in both A and B) and *difference* (the set of all elements that are in A but not in B). Many set operations combine two sets to produce a new set as their result.

Set operations are often depicted by drawings called Venn diagrams, which depict sets as circles and set operations as shaded overlapping between the circles. [Figure 11.3](#) shows some examples.

Figure 11.3 Set operation Venn diagrams



You can write code to perform set operations by calling methods whose names end with “All” using the relevant pair of sets. [Table](#)

11.4  summarizes which methods correspond to which set operations.

Table 11.4 Common Set Operations, Given Two Sets A and B

For example, we could rewrite the `totalElements` code to use a union with the `addAll` method:

```
// returns the number of elements contained in both set1 and
set2
public static int totalElements(Set<String> set1, Set<String>
set2) {
    Set<String> union = new HashSet<>(set1);
    union.addAll(set2);
    return union.size();
}
```

It's important to note that the set operations in Java modify the existing sets on which you call them, rather than creating new sets for you. Notice that in the preceding code, we initialize a new `HashSet` that contains all the elements from `set1` and then add the contents of `set2` to the new set, rather than combining `set1` and `set2` directly. We do this because the caller might not want us to disturb the sets' original contents.

Set Case Study: Lottery

Consider the task of writing a lottery program. The program should randomly generate a winning lottery ticket, then prompt the player to enter lotto numbers. Depending on how many numbers match, the player wins various prizes.

Sets make excellent collections for storing the winning lotto numbers and the player's numbers. They prevent duplicates, and they allow us to efficiently test whether a number in one set exists in the other. These features will help us to count the number of winning numbers the player has entered.

The following code uses a `Random` object to initialize a set of six winning lottery numbers between 1 and 40. The code uses a `while` loop because the same number might be randomly generated more than once:

```
Set<Integer> winningNumbers = new TreeSet<>();  
  
Random r = new Random();  
  
while (winningNumbers.size() < 6) {  
    int number = r.nextInt(40) + 1;  
    winningNumbers.add(number);  
}
```

Once the program has generated the winning number set, we'll read the player's lottery numbers into a second set. To figure out how many numbers the player has chosen correctly, we could search the winning number set to see whether it contains each number from the ticket. However, a more elegant way to perform this test is to determine the intersection between the winning numbers set and the player's ticket set. The following code creates the intersection of the player's ticket and the winning numbers by copying the ticket and then removing any elements from it that aren't winning numbers:

```
// find the winning numbers from the user's ticket
Set<Integer> intersection = new TreeSet<>(ticket);
intersection.retainAll(winningNumbers);
```

Once we have the intersection, we can ask for its size to see how many of the player's numbers were winning numbers; we can then calculate the appropriate cash prize amount for the player on the basis of that number. (Our version starts with a \$100 prize and doubles that figure for each winning number.)

Here is a complete implementation of the lottery program. We've created a few static methods for structure and added a few constants to represent the number of numbers, maximum number, and lotto prize amounts:

```
1 // Plays a lottery game with the user, reading
2 // the user's numbers and printing how many matched.
3
4 import java.util.*;
5
6 public class Lottery {
7     public static final int NUMBERS = 6;
8     public static final int MAX_NUMBER = 40;
9     public static final int PRIZE = 100;
10
11     public static void main(String[] args) {
12         // get winning number and ticket sets
13         Set<Integer> winning = createWinningNumbers();
14         Set<Integer> ticket = getTicket();
15         System.out.println();
16
17         // keep only winning numbers from user's ticket
18         Set<Integer> matches = new TreeSet<>(ticket);
19         matches.retainAll(winning);
20
21         // print results
22         System.out.println("Your ticket was: " + ticket);
23         System.out.println("Winning numbers: " + winning);
24         if (matches.size() > 0) {
25             double prize = PRIZE * Math.pow(2,
matches.size());
26             System.out.println("Matched numbers: " +
matches);
```

```
27         System.out.printf("Your prize is $%.2f\n",
prize);
28     }
29 }
30
31 // generates a set of the winning lotto numbers
32 public static Set<Integer> createWinningNumbers() {
33     Set<Integer> winning = new TreeSet<>();
34     Random r = new Random();
35     while (winning.size() < NUMBERS) {
36         int number = r.nextInt(MAX_NUMBER) + 1;
37         winning.add(number);
38     }
39     return winning;
40 }
41
42 // reads the player's lottery ticket from the console
43 public static Set<Integer> getTicket() {
44     Set<Integer> ticket = new TreeSet<>();
45     Scanner console = new Scanner(System.in);
46     System.out.print("Type " + NUMBERS + " lotto
numbers: ");
47     while (ticket.size() < NUMBERS) {
48         int number = console.nextInt();
49         ticket.add(number);
50     }
51     return ticket;
```

```
52 }
```

```
53 }
```

Here's one example output from running the program:

```
Type 6 lotto numbers: 2 8 15 18 21 32
```

```
Your ticket was: [2, 8, 15, 18, 21, 32]
```

```
Winning numbers: [1, 3, 15, 16, 18, 39]
```

```
Matched numbers: [15, 18]
```

```
Your prize is $400.00
```

11.3 Maps



Consider the task of writing a telephone book program that allows users to type a person's name and search for that person's phone number. You could store the data in an array, a list, or a set. Perhaps you'd make a small class called `PhoneBookRecord` that stores a person's name and phone number and a list to contain the `PhoneBookRecord` objects. When you want to search for a phone number, you'd traverse the list, looking for the `PhoneBookRecord` that matches the name entered by the user, and return the associated phone number.

A solution such as the one just described isn't very practical. If the list of records is large, it would take the program a long time to look at each one to find the right record to retrieve the phone number.

In many data-processing tasks, it's useful to link pairs of objects (such as a name and a telephone number). We often find ourselves saying, "I'd like to associate every A with a B." Perhaps we'd like to associate names with addresses so that when the user types a name, we can quickly look up that person's address. Or perhaps we want to count the number of occurrences of every word in a book by associating each word with its count of occurrences.

The abstract data type *map* describes a collection that allows you to create one-way associations between pairs of objects to solve these kinds of problems.

Map

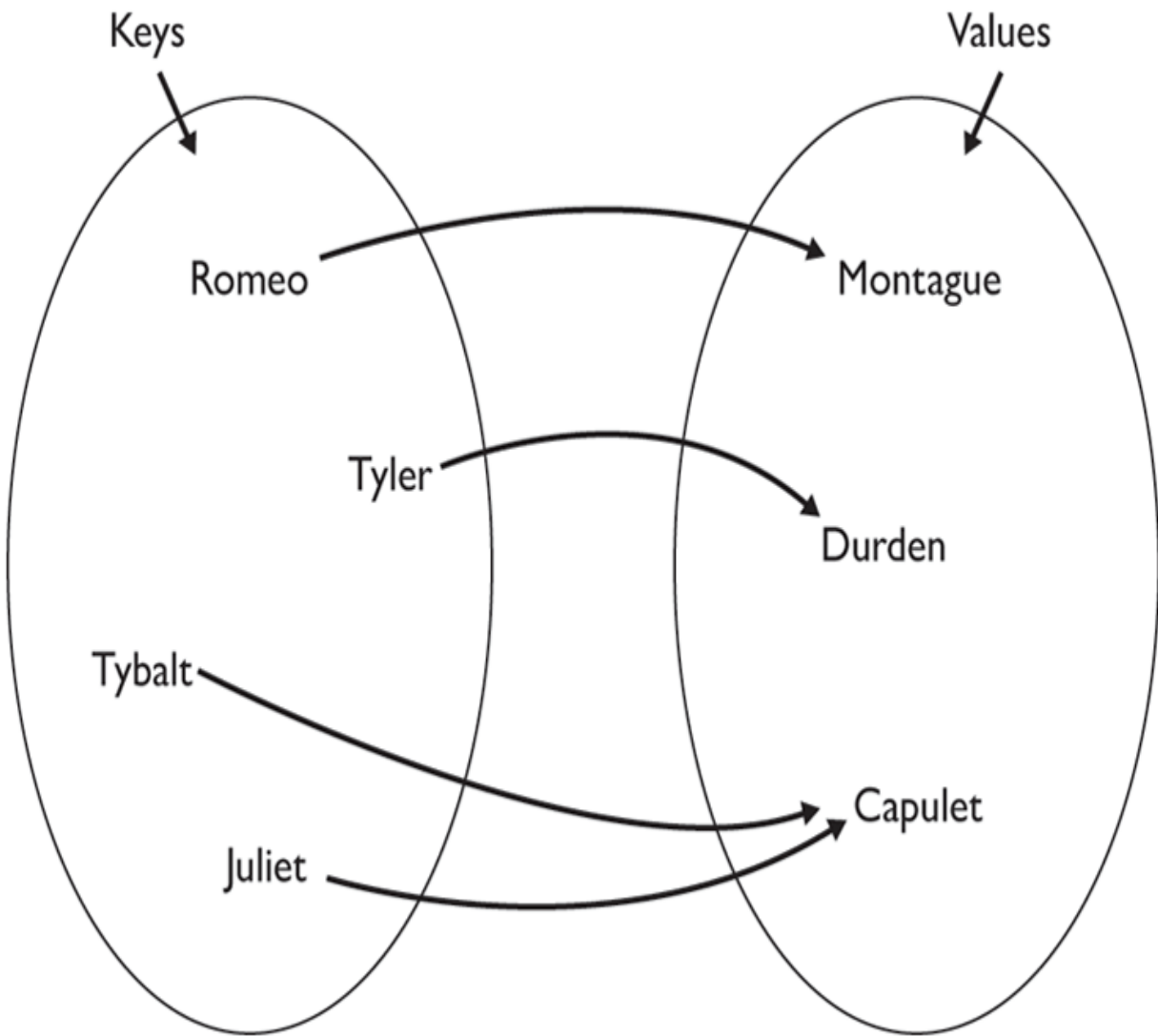
A collection that associates objects called keys with objects called values.

Maps can be used to solve a surprisingly large number of problems. A map can group all the words in a book by length and report how many words there are of each length. Maps can associate chat users with their set of friends and buddies. Maps can even represent a family tree associating each person with his or her mother and father.

A map associates *keys* with *values*. You can store a key/value pair into a map; later in your code, if you supply just the key to the map, it will give you back the value associated with that key. A key can map to only one value, but it's possible for multiple keys to map to the same value. The Java Collections Framework includes an interface called `Map` representing this ADT.

You can think of a map as a pair of connected collections: a set of keys and a collection of values associated with those keys. **Figure 11.4**  is an example of this idea that maps first names to last names.

Figure 11.4 Mapping keys (first names) to values (last names)



Basic Map Operations

The two primary classes that implement the `Map` interface are called `HashMap` and `TreeMap`. These class names parallel the ones used in the set collections, because the maps and sets have similar internal

implementations. `HashMap` is the more general-purpose map; a `TreeMap` stores comparable keys in sorted order.

A `Map` is constructed with not one but two generic type parameters, separated by a comma. The first type parameter represents the type of the keys, and the second represents the type of the values. The inclusion of two parameters makes the declaration line in your code lengthy. The line of code that follows is an example of constructing a salary map that associates people's names with their salaries. Notice that we have to use the wrapper type `Double` rather than the primitive type `double`:

```
Map<String, Double> salaryMap = new HashMap<>();
```

Notice the use of the diamond operator as discussed in [Chapter 10](#). The operator will infer the type of the keys and values from the two types given on the left side of the statement.

Key/value pairings are added to a map using its `put` method, which is roughly similar to the `add` method of most other collections. The `put` method accepts a key and a value as parameters and stores a mapping between the key and value in the map. If the key was previously associated with some other value, the new association replaces the old one. We can add key/value pairs to our salary map using code like the following:

```
salaryMap.put("Stuart Reges", 10000.00);  
salaryMap.put("Marty Stepp", 95500.00);  
salaryMap.put("Jenny", 86753.09);
```

Once you've added key/value pairs to a map, you can look up a value later by calling the map's `get` method, which accepts a key as a parameter and returns the value associated with that key:

```
double jenSalary = salaryMap.get("Jenny");  
System.out.printf("Jenny's salary is $%.2f\n", jenSalary);
```

To see whether a map contains a mapping for a given key, you can use the `containsKey` method, or you can call the `get` method and test for a `null` result:

```
Scanner console = new Scanner(System.in);  
System.out.print("Type a person's name: ");  
String name = console.nextLine();  
// search the map for the given name  
if (salaryMap.containsKey(name)) {  
    double salary = salaryMap.get(name);  
    System.out.printf("%s's salary is $%.2f\n", name, salary);  
} else {  
    System.out.println("I don't have a record for " + name);  
}
```

Table 11.5  lists several useful `Map` methods.

Table 11.5 Useful Methods of Maps

A `Map`'s `toString` method displays a comma-separated list of its key/value pairs. The order in which the keys appear depends on the type of map used, which we'll discuss in a moment. Here's what the salary map declared previously would look like when the program prints it:

```
{Jenny=86753.09, Stuart Reges=10000.0, Marty Stepp=95500.0}
```

Map Views (`keySet` and `values`)

Unlike most collections, a map doesn't have an `iterator` method, because it wouldn't be clear what you wanted to examine. The keys? The values? Both? Instead, maps have a pair of methods called `keySet` and `values` that respectively return a `Set` of all keys in the map and a `Collection` of all values in the map. These are sometimes called *collection views* of a map because each is a collection that exists conceptually within the map.

For example, consider a map that associates people's social security numbers with their names. In other words, the map's keys are nine-digit social security numbers and its values are names. We could create the map with the following code:

```
Map<Integer, String> ssnMap = new HashMap<>();  
ssnMap.put(867530912, "Jenny");  
ssnMap.put(239876305, "Stuart Reges");  
ssnMap.put(504386382, "Marty Stepp");
```

If we wanted to write a loop that printed the social security numbers of every person in the map, we could then call the `keySet` method on the map. This method returns a `Set` containing every key from the hash table—in this case, every `int` for a person's number. If you store the `keySet` in a variable, you should declare that variable as type `Set`, with the type of the map's keys between the `<` and `>` brackets:

```
Set<Integer> ssnSet = ssnMap.keySet();  
for (int ssn : ssnSet) {  
    System.out.println("SSN: " + ssn);  
}
```

The preceding code would produce the following output (the keys are in an unpredictable order since a `HashMap` is used):

```
SSN: 239876305  
SSN: 867530912  
SSN: 504386382
```

If we instead wanted to loop over every name (every value) stored in the map, we'd call the `values` method on the map. The `values` method returns a reference of type `Collection`, not of type `Set`, because the values may contain duplicates (it's legal for two keys to map to the same value). If you store the `values` result in a variable, you should declare that variable as type `Collection`, with the type of the map's values between the `<` and `>`:

```
Collection<String> names = ssnMap.values();  
for (String name : names) {  
    System.out.println("name: " + name);  
}
```

The preceding code would produce output such as the following:

```
name: Stuart Reges  
name: Jenny  
name: Marty Stepp
```

You can combine the keys and the values by looping over the keys and then getting the value for each key:

```
for (int ssn : ssnMap.keySet()) {  
    String name = ssnMap.get(ssn);  
    System.out.println(name + "'s SSN is " + ssn);  
}
```

Notice that this code doesn't declare a variable to store the key set, but instead calls `keySet` directly in the for-each loop. The code produces the following output:

```
Stuart Reges's SSN is 239876305  
Jenny's SSN is 867530912  
Marty Stepp's SSN is 504386382
```

A related method called `entrySet` returns objects of a type called `Map.Entry` that represents key/value pairs, but we won't explore this method here.

TreeMap versus HashMap

Just as there are two set implementations, `HashSet` and `TreeSet`, there are two flavors of `Map` collections in Java: `HashMap` and `TreeMap`. A `HashMap` performs a bit faster than a `TreeMap` and can store any type of data, but it keeps its keys in a somewhat haphazard order. A `TreeMap` can store only comparable data and performs a bit slower, but it keeps its keys in sorted order.

We could store the social security number information from the previous section in a `TreeMap` as follows:

```
Map<Integer, String> ssnMap = new TreeMap<>();  
ssnMap.put(867530912, "Jenny");  
ssnMap.put(239876305, "Stuart Reges");  
ssnMap.put(504386382, "Marty Stepp");  
System.out.println(ssnMap);
```

The keys would be ordered differently, leading to the following output when printing the map:

```
{239876305=Stuart Reges, 504386382=Marty Stepp,  
867530912=Jenny}
```

Notice that the social security numbers (the map's keys) are sorted in numeric order. This sorting can be useful for certain applications, but `HashMap` is still recommended for general use over `TreeMap`. Many

applications don't care about the order of the keys and benefit from the better performance of `HashMap`. `HashMap` works even on data that does not have a natural ordering.

Map Case Study: `WordCount`

In an earlier example, we counted the number of unique words in the book *Moby Dick*. What if we wanted to find the words that occur most frequently in the book? To do this, we should count how many times each word in the book occurs, then examine all of those counts and print the ones with the largest values.

Maps are very useful for solving these kinds of problems. We can create a word-count map in which each key is a word and its associated value is the number of occurrences of that word in the book:

```
wordCountMap = empty.  
for (each word from file) {  
    if (I have never seen this word before) {  
        set this word's count to 1.  
    } else {  
        increase this word's count by one.  
    }  
}
```

We'll need a `Scanner` to read the appropriate file and a `Map` to store the word counts. We'll use a `TreeMap` so that the frequently used words will be shown in alphabetical order:

```
Map<String, Integer> wordCountMap = new TreeMap<>();  
Scanner in = new Scanner(new File("mobydick.txt"));
```

We can now read the file's contents and store each word that the program encounters in the map. If we come across a word that we have already seen, we retrieve its old `count` value, increment it by 1, and put the new value back into the map. Recall that when you put a key/value mapping into a map that already contains that key, the old mapping is replaced. For example, if the word `"ocean"` was mapped to the number 25 and we put in a new mapping from `"ocean"` to 26, the old mapping from `"ocean"` to 25 would be replaced; we don't have to remove it manually. Here's the code to build up the map:

```
while (in.hasNext()) {  
    String word = in.next().toLowerCase();  
    if (wordCountMap.containsKey(word)) { // seen before  
        int count = wordCountMap.get(word);  
        wordCountMap.put(word, count + 1);  
    } else { // never seen before  
        wordCountMap.put(word, 1);  
    }  
}
```

Once we've built the word-count map, if we want to print all words that appear more than, say, 2,000 times in the book, we can write code like the following:

```
for (String word : wordCountMap.keySet()) {  
    int count = wordCountMap.get(word);  
    if (count > 2000) {  
        System.out.println(word + " occurs " + count + "  
times.");  
    }  
}
```

Here's the complete program, with a method added for structure and a constant added for the number of occurrences that are needed for a word to be considered among the most frequent:

```
1  // Uses maps to implement a word count, so that the user  
2  // can see which words occur the most in the book Moby-  
Dick.  
3  
4  import java.io.*;  
5  import java.util.*;  
6  
7  public class WordCount {  
8      // minimum number of occurrences needed to be printed
```

```
9      public static final int OCCURRENCES = 2000;
10
11      public static void main(String[] args)
12          throws FileNotFoundException {
13          System.out.println("This program displays the
most");
14          System.out.println("frequently occurring words
from");
15          System.out.println("the book Moby Dick.");
16          System.out.println();
17
18          // read the book into a map
19          Scanner in = new Scanner(new File("mobydick.txt"));
20          Map<String, Integer> wordCountMap =
getCountMap(in);
21
22          for (String word: wordCountMap.keySet()) {
23              int count = wordCountMap.get(word);
24              if (count > OCCURRENCES) {
25                  System.out.println(word + " occurs " +
26                      count + " times.");
27              }
28          }
29      }
30
31      // Reads book text and returns a map from words to
counts.
32      public static Map<String, Integer> getCountMap(Scanner
```

```

in) {
33      Map<String, Integer> wordCountMap = new TreeMap<>
();
34
35      while (in.hasNext()) {
36          String word = in.next().toLowerCase();
37          if (wordCountMap.containsKey(word)) {
38              // seen this word before; increment count
39              int count = wordCountMap.get(word);
40              wordCountMap.put(word, count + 1);
41          } else {
42              // never seen this word before
43              wordCountMap.put(word, 1);
44          }
45      }
46
47      return wordCountMap;
48  }
49  }

```

The program produces the following output for *Moby-Dick*:

```

This program displays the most
frequently occurring words from
the book Moby-Dick.
a occurs 4509 times.

```

```
and occurs 6138 times.  
his occurs 2451 times.  
in occurs 3975 times.  
of occurs 6405 times.  
that occurs 2705 times.  
the occurs 13991 times.  
to occurs 4433 times.
```

Collection Overview

We've discussed three major abstract data types in this chapter: lists, sets, and maps. It's important to understand the differences between them and to know when each should be used. **Table 11.6**  summarizes the different types of collections and their pros and cons.

Table 11.6 Comparison of Lists, Sets, and Maps

When you approach a new programming problem involving data, you can ask yourself questions to decide what type of collection is most appropriate. Here are some examples:

- What are you going to do with the data? Do you intend to add and remove many elements, search the data many times, or connect this data to other data?

- Do you plan to search the data? If so, how? If you want to examine elements in order of insertion, you want a list. If you intend to search for arbitrary elements, a set may be better. If you need to find an object, given partial information about it (for example, a PIN or ID number, to find a user's bank account object), a map may be best.
- In what order should the elements be stored? Lists hold their elements in order of insertion, whereas the tree collections (`TreeSet` and `TreeMap`) order elements by their natural ordering. If order doesn't matter, you may want a hash table collection such as a `HashSet`.

Chapter Summary

A collection is an object that stores a group of other objects.

Examples of collections are `ArrayList`, `HashSet`, and `TreeMap`.

Collections are used to structure, organize, and search data.

A linked list is a collection that's similar to an `ArrayList` but that is implemented internally by storing each element in a small container object called a node. Linked lists can perform certain operations faster than array lists, such as adding data to and removing data from the front or middle of the list.

An iterator is an object that keeps track of the current position in a list and expedites the examination of its elements in sequential order. Linked lists are often used with iterators for increased efficiency.

An abstract data type (ADT) is a specification of the operations that can be performed on data. Two examples of ADTs are `List` and `Set`. ADTs in the Java Collections Framework are represented as interfaces (e.g., the `List` interface, which is implemented both by `LinkedList` and by `ArrayList`).

A set is a collection that doesn't allow duplicates. Sets generally can be searched very quickly to see whether they contain a particular element value. The `Set` interface represents sets.

There are two major set classes in Java: `TreeSet` and `HashSet`. A `TreeSet` holds `Comparable` data in a sorted order; a `HashSet` can hold any data and can be searched faster, but its elements are stored in an unpredictable order.

A map is a collection that associates key objects with value objects. Maps are used to create relationships of association between pieces of data, such as a person's name and phone number.

There are two major map classes in Java: `TreeMap` and `HashMap`. A `TreeMap` holds `Comparable` keys in a sorted order; a `HashMap` can hold any data as its keys and performs value lookups faster, but its keys are stored in an unpredictable order.

Self-Check Problems

Section 11.1: Lists

1. When should you use a `LinkedList` instead of an `ArrayList`?
2. Would a `LinkedList` or an `ArrayList` perform better when run on the following code? Why?

```
public static int min(List<Integer> list) {  
    int min = list.get(0);  
    for (int i = 1; i < list.size(); i++) {  
        if (list.get(i) < min) {  
            min = list.get(i);  
        }  
    }  
    return min;  
}
```

3. What is an iterator? Why are iterators often used with linked lists?
4. Write a piece of code that counts the number of duplicate elements in a linked list, that is, the number of elements whose values are repeated at an earlier index in the list. Assume that all duplicates in the list occur consecutively. For example, the list `[1, 1, 3, 5, 5, 5, 5, 7, 7, 11]` contains five duplicates: one duplicate of element value `1`, three duplicates of element value `5`, and one duplicate of element value `7`.

5. Write a piece of code that inserts a `String` into an ordered linked list of `Strings`, maintaining sorted order. For example, for the list `[" Alpha" , " Baker" , " Foxtrot" , " Tango" , " Whiskey" ]`, inserting `"Charlie"` in order would produce the list `[" Alpha" , " Baker" , " Charlie" , " Foxtrot" , " Tango" , " Whiskey" ]`.
6. Write a method called `removeAll` that accepts a linked list of integers as a parameter and removes all occurrences of a particular value. You must preserve the original relative order of the remaining elements of the list. For example, the call `removeAll(list, 3)` would change the list `[3, 9, 4, 2, 3, 8, 17, 4, 3, 18, 2, 3]` to `[9, 4, 2, 8, 17, 4, 18, 2]`.
7. Write a method called `wrapHalf` that accepts a linked list of integers as a parameter and moves the first half of the list to the back of the list. If the list contains an odd number of elements, the smaller half is wrapped (in other words, for a list of size N , the middle element, $N/2$, becomes the first element in all cases). For example, calling `wrapHalf` on the list `[1, 2, 3, 4, 5, 6]`, would change that list into `[4, 5, 6, 1, 2, 3]`. For the list `[5, 6, 7, 8, 9]`, the result would be `[7, 8, 9, 5, 6]`.
8. What is an abstract data type (ADT)? What ADT does a linked list implement?
9. **Self-Check Problem 4**  asked you to write code that would count the duplicates in a linked list. Rewrite your code as a

method called `countDuplicates` that will allow either an `ArrayList` or a `LinkedList` to be passed as the parameter.

Section 11.2: Sets

10. A `List` has every method that a `Set` has, and more. So why would you use a `Set` rather than a `List`?
11. When should you use a `TreeSet`, and when should you use a `HashSet`?
12. A `Set` doesn't have the `get` and `set` methods that an `ArrayList` has. How do you examine every element of a `Set`?
13. What elements are contained in the following set after this code executes?

```
Set<Integer> set = new HashSet<>();  
set.add(74);  
set.add(12);  
set.add(74);  
set.add(74);  
set.add(43);  
set.remove(74);  
set.remove(999);  
set.remove(43);  
set.add(32);  
set.add(12);  
set.add(9);  
set.add(999);
```

14. How do you perform a union operation on two sets? An intersection? Try to give an answer that doesn't require any loops.
15. Write the output produced when the following method is passed each of the following lists:

```
public static void mystery(List<String> list) {  
    Set<String> result = new TreeSet<>();  
    for (String element : list) {  
        if (element.compareTo(list.get(0)) < 0) {  
            result.add(element);  
        } else {  
            result.clear();  
        }  
    }  
    System.out.println(result);  
}
```

- a. [marty, stuart, helene, jessica, amanda]
- b. [sara, caitlin, janette, zack, riley]
- c. [zorah, alex, tyler, roy, roy, charlie, phil, charlie, tyler]

Section 11.3: Maps

16. Write the code to declare a `Map` that associates people's names with their ages. Add mappings for your own name and age, as well as those of a few friends or relatives.
17. A `Map` doesn't have the `get` and `set` methods that an `ArrayList` has. It doesn't even have an iterator method like a `Set` does, nor can you use a for-each loop on it directly. How do you examine every key (or every value) of a `Map`?
18. What keys and values are contained in the following map after this code executes?

```
Map<Integer, String> map = new HashMap<>();  
map.put(8, "Eight");  
map.put(41, "Forty-one");  
map.put(8, "Ocho");  
map.put(18, "Eighteen");  
map.put(50, "Fifty");  
map.put(132, "OneThreeTwo");  
map.put(28, "Twenty-eight");  
map.put(79, "Seventy-nine");  
map.remove(41);  
map.remove(28);  
map.remove("Eight");  
map.put(50, "Forty-one");  
map.put(28, "18");  
map.remove(18);
```

19. Write the output produced when the following method is passed each of the following maps:

```
public static void mystery(Map<String, String> map) {  
    Map<String, String> result = new TreeMap<>();  
    for (String key : map.keySet()) {  
        if (key.compareTo(map.get(key)) < 0) {  
            result.put(key, map.get(key));  
        } else {  
            result.put(map.get(key), key);  
        }  
    }  
    System.out.println(result);  
}
```

- a. {two=deux, five=cinq, one=un, three=trois,
four=quatre}
- b. {skate=board, drive=car, program=computer,
play=computer}
- c. {siskel=ebert, girl=boy, H=T, ready=begin, first=last,
begin=end}
- d. {cotton=shirt, tree=violin, seed=tree, light=tree,
rain=cotton}

20. Write the output produced when the following method is passed each of the following maps:


```

map2) {
    Map<String, String> result = new TreeMap<>();
    for (String s1 : map1.keySet()) {
        if (map2.containsKey(map1.get(s1))) {
            result.put(s1, map2.get(map1.get(s1)));
        }
    }
    return result;
}

```

- a. map1: {bar=1, baz=2, foo=3, mumble=4},
map2: {1=earth, 2=wind, 3=air, 4=fire}
- b. map1: {five=105, four=104, one=101, six=106,
three=103, two=102},
map2: {99=uno, 101=dos, 103=tres, 105=quattro}
- c. map1: {a=42, b=9, c=7, d=15, e=11, f=24, g=7},
map2: {1=four, 3=score, 5=and, 7=seven, 9=years,
11=ago}

22. Modify the `WordCount` program so that it prints the most frequently occurring words sorted by number of occurrences. To do this, write code at the end of the program to create a reverse map from counts to words that is based on the original map. Assume that no two words of interest occur the exact same number of times.

Exercises

1. Modify the `Sieve` program developed in [Section 11.1](#) to make two optimizations. First, instead of storing all integers up to the maximum in the `numbers` list, store only 2 and all odd numbers from 3 upward. Second, write code to ensure that if the first number in the `numbers` list ever reaches the square root of the maximum, all remaining values from the `numbers` list are moved into the `primes` list. (Why is this a valid operation?)
2. Write a method called `alternate` that accepts two `Lists` as its parameters and returns a new `List` containing alternating elements from the two lists, in the following order:
 - First element from first list
 - First element from second list
 - Second element from first list
 - Second element from second list
 - Third element from first list
 - Third element from second list
 - ...

If the lists do not contain the same number of elements, the remaining elements from the longer list should be placed consecutively at the end. For example, for a first list of `[1, 2, 3, 4, 5]` and a second list of `[6, 7, 8, 9, 10, 11, 12]`, a call

of `alternate(list1, list2)` should return a list containing `[1, 6, 2, 7, 3, 8, 4, 9, 5, 10, 11, 12]`.

3. Write a method called `removeInRange` that accepts four parameters: a `LinkedList`, an element value, a starting index, and an ending index. The method's behavior is to remove all occurrences of the given element that appear in the list between the starting index (inclusive) and the ending index (exclusive). Other values and occurrences of the given value that appear outside the given index range are not affected. For example, for the list `[0, 0, 2, 0, 4, 0, 6, 0, 8, 0, 10, 0, 12, 0, 14, 0, 16]`, a call of `removeInRange(list, 0, 5, 13)` should produce the list `[0, 0, 2, 0, 4, 6, 8, 10, 12, 0, 14, 0, 16]`. Notice that the zeros located at indexes between 5 inclusive and 13 exclusive in the original list (before any modifications were made) have been removed.
4. Write a method called `partition` that accepts a list of integers and an integer value `E` as its parameter, and rearranges (partitions) the list so that all the elements with values less than `E` occur before all elements with values greater than `E`. The exact order of the elements is unimportant, so long as all elements less than `E` appear before all elements greater than `E`. For example, for the linked list `[15, 1, 6, 12, -3, 4, 8, 21, 2, 30, -1, 9]`, one acceptable ordering of the list after a call of `partition(list, 5)` would be `[-1, 1, 2, 4, -3, 12, 8, 21, 6, 30, 15, 9]`. You may assume that the list contains no duplicates and does not contain the element value `E`.

5. Write a method called `sortAndRemoveDuplicates` that accepts a list of integers as its parameter and rearranges the list's elements into sorted ascending order, as well as removing all duplicate values from the list. For example, the list `[7, 4, -9, 4, 15, 8, 27, 7, 11, -5, 32, -9, -9]` would become `[-9, -5, 4, 7, 8, 11, 15, 27, 32]` after a call to your method. Use a `Set` as part of your solution.
6. Write a method `countUnique` that accepts a list of integers as a parameter and returns the number of unique integer values in the list. Use a set as auxiliary storage to help you solve this problem. For example, if a list contains the values `[3, 7, 3, -1, 2, 3, 7, 2, 15, 15]`, your method should return `5`. The empty list contains `0` unique values.
7. Write a method `countCommon` that accepts two lists of integers as parameters and returns the number of unique integers that occur in both lists. Use one or more sets as storage to help you solve this problem. For example, if one list contains the values `[3, 7, 3, -1, 2, 3, 7, 2, 15, 15]` and the other list contains the values `[-5, 15, 2, -1, 7, 15, 36]`, your method should return `4` because the elements `-1, 2, 7,` and `15` occur in both lists.
8. Write a method `maxLength` that accepts a set of strings as a parameter and that returns the length of the longest string in the list. If your method is passed an empty set, it should return `0`.

9. Write a method `hasOdd` that accepts a set of integers as a parameter and returns `true` if the set contains at least one odd integer and `false` otherwise. If passed the empty set, your method should return `false`.
10. Write a method `removeEvenLength` that accepts a set of strings as a parameter and that removes all of the strings of even length from the set.
11. Write a method called `symmetricSetDifference` that accepts two `Sets` as parameters and returns a new `Set` containing their symmetric set difference (that is, the set of elements contained in either of the two sets but not in both). For example, the symmetric difference between the sets `[1, 4, 7, 9]` and `[2, 4, 5, 6, 7]` is `[1, 2, 5, 6, 9]`.
12. Write a method `contains3` that accepts a list of strings as a parameter and returns `true` if any single string occurs at least 3 times in the list, and `false` otherwise. Use a map as auxiliary storage.
13. Write a method `isUnique` that accepts a map whose keys and values are strings as a parameter and returns `true` if no two keys map to the same value (and `false` if any two or more keys do map to the same value). For example, if the map contains the following key/value pairs, your method would return `true`: `{Marty=Stepp, Stuart=Reges, Jessica=Miller, Amanda=Camp, Hal=Perkins}`. But calling it on the following map would return `false`, because of two mappings for `Perkins` and

```
Reges: {Kendrick=Perkins, Stuart=Reges, Jessica=Miller,  
Bruce=Reges, Hal=Perkins}.
```

14. Write a method `intersect` that accepts two maps whose keys are strings and whose values are integers as parameters and returns a new map containing only the key/value pairs that exist in both of the parameter maps. In order for a key/value pair to be included in your result, not only do both maps need to contain a mapping for that key, but they need to map it to the same value. For example, if the two maps passed are

```
{Janet=87, Logan=62, Whitaker=46, Alyssa=100, Stefanie=80,
```

```
Jeff=88, Kim=52, Sylvia=95} and {Logan=62, Kim=52,
```

```
Whitaker=52, Jeff=88, Stefanie=80, Brian=60, Lisa=83,
```

```
Sylvia=87}, your method would return the following new map
```

(the order of the key/value pairs does not matter): `{Logan=62,`

```
Stefanie=80, Jeff=88, Kim=52}.
```

15. Write a method `maxOccurrences` that accepts a list of integers as a parameter and returns the number of times the most frequently occurring integer (the “mode”) occurs in the list. Solve this problem using a map as auxiliary storage. If the list is empty, return `0`.

16. Write a method called `is1to1` that accepts a map whose keys and values are strings as its parameter and returns `true` if no two keys map to the same value. For example, `{Marty=206-9024, Hawking=123-4567, Smith=949-0504, Newton=123-4567}` should return `false`, but `{Marty=206-9024, Hawking=555-1234,`

`Smith=949-0504, Newton=123-4567}` should return `true`. The empty map is considered 1-to-1 and returns `true`.

17. Write a method called `subMap` that accepts two maps from strings to strings as its parameters and returns `true` if every key in the first map is also contained in the second map and maps to the same value in the second map. For example, `{Smith=949-0504, Marty=206-9024}` is a submap of `{Marty=206-9024, Hawking=123-4567, Smith=949-0504, Newton=123-4567}`.

The empty map is a submap of every map.

18. Write a method called `reverse` that accepts a map from strings to strings as a parameter and returns a new map that is the reverse of the original. The reverse of a map is a new map that uses the values from the original as its keys and the keys from the original as its values. Since a map's values need not be unique but its keys must be, you should have each value map to a set of keys. In other words, if the original map maps keys of type K to values of type V , the new map should map keys of type V to values that are `Sets` containing elements of type K . For example, the map `{42 = Marty, 81 = Sue, 17 = Ed, 31=Dave, 56=Ed, 3=Marty, 29 = Ed}` has a reverse of `{Marty=[42, 3], Sue = [81], Ed = [17, 56, 29], Dave = [31]}`. (The order of the keys and values does not matter.)

19. Write a method called `rarest` that accepts a map whose keys are strings and whose values are integers as a parameter and returns the integer value that occurs the fewest times in the

map. If there is a tie, return the smaller integer value. If the map is empty, throw an exception.

20. Write a modified version of the `Vocabulary` program developed in [Chapter 10](#)  that uses sets rather than `ArrayLists` to store its words. (The program will be noticeably shorter and will run faster!)

21. Write a method called `pairCounts` that accepts a list of strings representing individual words and counts the number of occurrences of all 2-character sequences of letters in those words. For example, suppose the list contains `["banana", "bends", "i", "mend", "sandy"]`. This list contains the following two-character pairs: `"ba", "an", "na", "an", "na"` from `"banana"`; `"be", "en", "nd", "ds"` from `"bends"`; `"me", "en", "nd"` from `"mend"`; and `"sa", "an", "nd", "dy"` from `"sandy"`. (Note that `"i"` is only one character long, so it contains no pairs.) Your method should count the occurrences of these pairs of letters and return the counts as a `Map`. In this case you would return `{an=3, ba=1, be=1, ds=1, dy=1, en=2, me=1, na=2, nd=3, sa=1}`.

Notice that pairings that occur more than once in the same word should be counted as separate occurrences. For example, `"an"` and `"na"` each occur twice in `"banana"`.

Programming Projects

1. Write a program that computes the edit distance (also called the Levenshtein distance, for its creator Vladimir Levenshtein) between two words. The edit distance between two strings is the minimum number of operations that are needed to transform one string into the other. For this program, an operation is a substitution of a single character, such as from “brisk” to “brick”. The edit distance between the words “dog” and “cat” is 3, following the chain of “dot”, “cot”, and “cat” to transform “dog” into “cat”. When you compute the edit distance between two words, each intermediate word must be an actual valid word. Edit distances are useful in applications that need to determine how similar two strings are, such as spelling checkers.

Read your input from a dictionary text file. From this file, compute a map from every word to its immediate neighbors, that is, the words that have an edit distance of 1 from it. Once this map is built, you can walk it to find paths from one word to another.

A good way to process paths to walk the neighbor map is to use a linked list of words to visit, starting with the beginning word, such as “dog”. Your algorithm should repeatedly remove the front word of the list and add all of its neighbors to the end of the list, until the ending word (such as “cat”) is found or until

the list becomes empty, which indicates that no path exists between the two words.

2. Write a program that solves the classic “stable marriage” problem. This problem deals with a group of men and a group of women. The program tries to pair them up so as to generate as many stable marriages as possible. A set of marriages is unstable if you can find a man and a woman who would rather be married to each other than to their current spouses (in which case the two would be inclined to divorce their spouses and marry each other).

The input file for the program will list all of the men, one per line, followed by a blank line, followed by all of the women, one per line. The men and women are numbered according to their positions in the input file (the first man is #1, the second man is #2, and so on; the first woman is #1, the second woman is #2, and so on). Each input line (except for the blank line separating men from women) lists the person’s name, followed by a colon, followed by a list of integers. These integers are the marriage partner preferences of this particular person. For example, see the following input line in the men’s section:

```
Joe: 10 8 35 9 20 22 33 6 29 7 32 16 18 25
```

This line indicates that the person is named “Joe” and that his first choice for marriage is woman #10, his second choice is woman #8, and so on. Any women not listed are considered unacceptable to Joe.

The stable marriage problem is solved by the following algorithm:

```
assign each person to be free.
while (some man M with a nonempty preference list is
free) {
    W = first woman on M's list.
    if (some man P is engaged to W) {
        assign P to be free.
    }
    assign M and W to be engaged to each other.
    for (each successor Q of M who is on W's list) {
        delete W from Q's preference list.
        delete Q from W's preference list.
    }
}
```

Consider the following input:

```
Man 1: 4 1 2 3
Man 2: 2 3 1 4
Man 3: 2 4 3 1
Man 4: 3 1 4 2
Woman 1: 4 1 3 2
Woman 2: 1 3 2 4
Woman 3: 1 2 3 4
Woman 4: 4 1 3 2
```

The following is a stable marriage solution for this input:

Man 1 and Woman 4

Man 3 and Woman 2

Man 2 and Woman 3

Man 4 and Woman 1

3. Write a program that solves the classic “random writer” problem. This problem deals with reading input files of text and examining the frequencies of characters. On the basis of those frequencies, you can generate randomized output that appears to match the writing style of the original document. The longer the chains you link together, the more accurate the random text will sound. For example, level 4 random text (text with chains of 4 letters long) generated from *Tom Sawyer* might look like this: “en themself, Mr. Welshman, but him awoke, the balmy shore. I’ll give him that he couple overy because in the slated snufflindeed structure’s kind was rath. She said that the wound the door a fever eyes that WITH him.” Level 10 random text from the same source might look like this: “you understanding that they don’t come around in the cave should get the word beauteous was over-fondled, and that together and decided that he might as we used to do—it’s nobby fun. I’ll learn you.” Search the Internet for “Random Writer” to learn more about this problem, such as the specification posed by computer scientist Joseph Zachary.